Received [date], accepted [date], date of publication [date], date of current version [date]. Digital Object Identifier 10.1109/ACCESS.2026.DOI

AgriSense AI: Design and Implementation of an Edge-Authoritative IoT Platform for Fail-Safe Closed-Loop Irrigation and Leakage-Aware Crop Disease Diagnosis

TISHA JHARIYA[1], **AKIKO GAJBHIYE**[1], **ANSH MESHRAM**[1], **AGYA CHOURASIA**[1], AND **[GUIDE NAME]**[1]

[1]Department of Emerging Technologies (Artificial Intelligence & Data Science), S. B. Jain Institute of Technology, Management and Research, Nagpur, Maharashtra 441501, India
Corresponding author: [Guide Name] ([email address])

ABSTRACT Smart-agriculture systems that combine microcontroller sensing, automated irrigation and image-based disease diagnosis are widely reported, yet three engineering weaknesses recur: the irrigation decision usually lives in the cloud and stops when connectivity fails, a disconnected soil probe is often indistinguishable from saturated or bone-dry soil, and headline disease-classification accuracies are measured on laboratory imagery with splits that permit near-duplicate leakage. This paper presents the design, implementation and verification of AgriSense AI, an open three-tier platform that addresses each weakness explicitly. An ESP32 sensor node evaluates an ordered, seven-rule irrigation policy on the device itself, so that safety interlocks (failed probe, rain, saturated soil) dominate any operator or cloud command, with a 30/70 % hysteresis band, a 15 min run cap, a 30 min cooldown and an 11-case boot-time self-test. Telemetry and commands flow directly between the device and a managed PostgreSQL store with row-level security, so monitoring and manual control survive a backend outage. A demand-driven ESP32-CAM relay streams the field without port forwarding. Diagnosis is served by a two-stage ONNX pipeline built on a leakage-aware data path (SHA-256 and perceptual-hash de-duplication, duplicate-group-aware stratified splits, a field dataset held out for evaluation only), with a schema-constrained vision-language fallback and a telemetry-grounded advisory assistant. A YOLO11n-cls baseline with 1.57 M parameters reaches 99.32 % top-1 and 100 % top-5 accuracy on 10,861 PlantVillage validation images across 38 classes and runs in 6.07 ms per image on one CPU thread. We report that figure as a laboratory upper bound, not a field estimate, show analytically why the exported model cannot run on the base ESP32 as-is, and list the field measurements that remain open.

INDEX TERMS Internet of Things, precision agriculture, smart irrigation, fail-safe control, ESP32, plant disease classification, YOLO11, ONNX, data leakage, edge computing, large language models.

I. INTRODUCTION

Agriculture is under simultaneous pressure from water scarcity and crop disease. Irrigated agriculture accounts for roughly 70 % of global freshwater withdrawals [1], while pathogens and pests are estimated to cost 10–28 % of wheat, 25–41 % of rice and 20–41 % of maize yield worldwide, with the heaviest losses in food-deficit regions [2], [3]. For smallholders both problems share a cause: decisions are made by eye and by habit, on fixed schedules and late visual inspection, because sensing and expert diagnosis are expensive.

The Internet of Things (IoT) and deep learning have each been proposed as the remedy. Wireless soil-moisture sensing with automated actuation was demonstrated more than a decade ago [4] and has since been extended with machine-learning scheduling and platform-scale water management [5]–[7]. Convolutional networks trained on leaf images report accuracies above 99 % [8]–[10]. Systems that combine the two on low-cost microcontrollers are now common [11]–[13].

Building one more such system is not, by itself, a contribution. What we found missing when reviewing these systems was attention to how they *fail*. Three failure modes motivated this work:

1) **Cloud-dependent control.** When the pump decision runs on a server, a Wi-Fi drop or backend crash leaves the field either unwatered or, worse, with a pump latched on.
2) **Fail-unsafe sensing.** A disconnected analogue soil probe floats to an ADC value near zero. Depending on calibration direction this reads as saturated or as completely dry soil; in the latter case a naïve threshold rule runs the pump indefinitely.
3) **Unaudited accuracy.** Laboratory leaf datasets contain near-duplicate photographs of the same leaf; random splits place them on both sides of the partition, and the resulting accuracy does not transfer to field photographs, where published models fall to roughly 31–68 % [8], [14], [15].

This paper describes AgriSense AI, a complete, open implementation designed around these three failure modes, and reports what has and has not yet been measured. Its contributions are:

1) An **edge-authoritative irrigation controller** whose ordered rule set places sensor interlocks above operator and cloud commands, with hysteresis, hard run-time limits and a boot-time self-test (Section V); and a simulation showing its effect on actuator cycling under sensor noise (Section X-D).

2) A **cloud-decoupled data path** in which the device writes telemetry and reads commands directly from a row-level-secured database, so that the application backend is required only for value-added services and every optional service degrades independently (Sections III and VI).
3) A **leakage-aware diagnosis pipeline**: exact and perceptual-hash de-duplication, duplicate-group-aware stratified splitting, a field dataset quarantined for evaluation, and a two-stage species-then-condition classifier exported to ONNX with confidence gating (Section VII).
4) A **measured baseline and a static edge-feasibility bound**: training curves, model complexity, CPU latency, and a closed-form memory analysis of the exported model against ESP32 SRAM (Section X).
5) An **explicit limitations statement** separating implemented, measured and pending items (Section XI).

Section II reviews related work. Sections III–IX describe the architecture, hardware, firmware, backend, diagnosis pipeline, advisory layer and client applications. Section X reports results, Section XI discusses limitations, and Section XII concludes.

II. RELATED WORK

A. IOT-BASED IRRIGATION

Gutiérrez et al. [4] combined a wireless sensor network with a GPRS gateway and threshold-based actuation. Goap et al. [5] added weather forecasts and machine-learning prediction of soil moisture to schedule irrigation, and the SWAMP project [6] generalised this to a platform for precision irrigation. Reviews [7], [16] report water savings spanning roughly 9–50 %, a range wide enough that unmetered claims are uninformative; a recent metered trial reports a 27.16 % reduction (431.47 L vs. 592.39 L) against a fixed schedule [17]. Few of these works specify behaviour under sensor failure or connectivity loss, which is the focus of our controller design.

B. DEEP LEARNING FOR PLANT DISEASE DIAGNOSIS

Following Mohanty et al. [8], transfer learning from ImageNet-pretrained backbones [18], [19] became the standard recipe on the PlantVillage corpus [20], with accuracies above 99 % [9], [10], [21]. The same authors observed a fall to about 31 % on images from outside the corpus, and PlantDoc [22] was assembled to provide a field-condition benchmark; the current cross-domain reference point for PlantVillage-trained models on such data is about 68 % [14]. Barbedo attributes the gap to background clutter, capture variability and lesion co-occurrence absent from laboratory images [23]. We adopt the YOLO11 classification family [24] for its maintained export path to ONNX.

C. EDGE AND TINYML DEPLOYMENT

Microcontroller inference is bound by on-chip SRAM for activations rather than flash for weights [25], [26], and relies on integer quantization [27] executed by embedded runtimes such as TensorFlow Lite Micro [28]. Published ESP32-class results disagree: Nyakuri et al. found ESP32-CAM unable to host their model and moved to a Raspberry Pi 5 [12], whereas Samanta et al. report 92 % on ESP32-CAM [11] and Aqasizade et al. 94.6 % on ESP32-S3 with a seven-day field deployment [13].

D. LANGUAGE MODELS IN AGRICULTURE

Large language models (LLMs) have been evaluated as extension agents [29] and on agronomy examinations [30]; retrieval augmentation and fine-tuning both improve domain answers [31], [32]. A consistent caution is that ungrounded answers can be confidently wrong, which motivates grounding the model in measured farm state, as done here.

TABLE 1. Positioning against representative systems. n/s = not stated in the cited source; — = not applicable. AgriSense is behind on field validation.

System	Control locus	Fail-safe probe	Disease model	Field-validated	LLM advisor
Gutiérrez [4]	Gateway	n/s	No	Yes (irrigation)	No
Goap [5]	Cloud/ML	n/s	No	Yes (irrigation)	No
Samanta [11]	—	—	ESP32-CAM, 92 %	No	No
Nyakuri [12]	—	—	RPi 5, 98.6 %	n/s	No
Aqasizade [13]	—	—	ESP32-S3, 94.6 %	Yes (7 days)	No
AgriSense (this work)	**On-node**	**Yes**	Server ONNX, 99.32 % lab	**Pending**	**Yes**

III. SYSTEM ARCHITECTURE

AgriSense is organised in three tiers (Fig. 1). The **field tier** contains two physically separate boards: an ESP32 DevKit V1 sensor/actuator node and an AI-Thinker ESP32-CAM imaging node. The **cloud tier** consists of a managed PostgreSQL database with authentication, Realtime subscriptions and row-level security (Supabase [33]), and a Python 3.11 FastAPI backend packaged with Docker. The **client tier** comprises a web dashboard and a cross-platform mobile application.

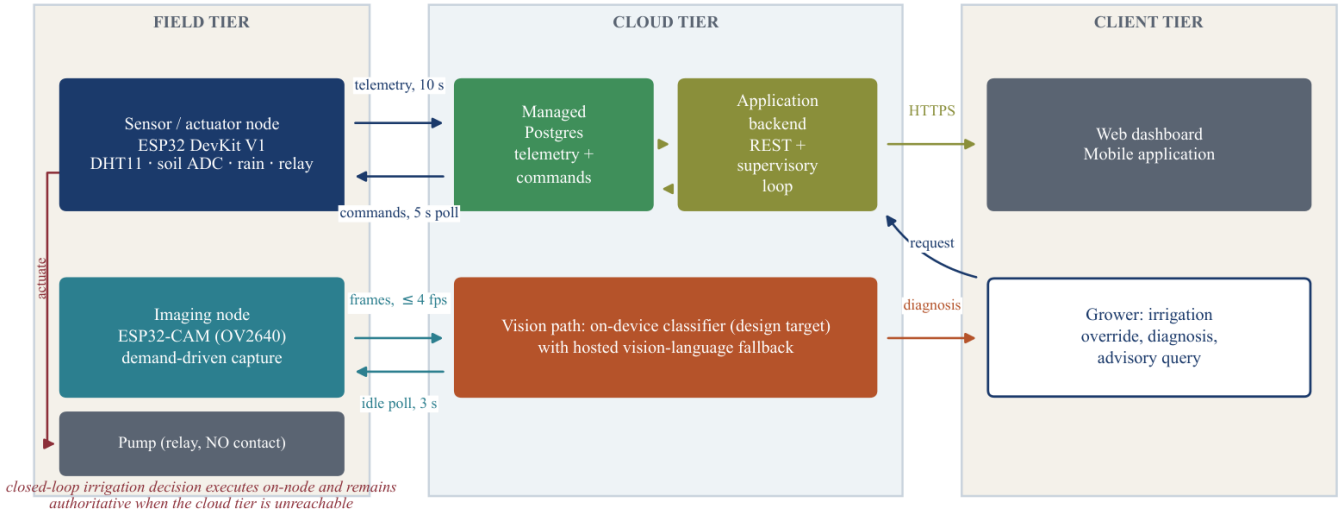


FIGURE 1. Three-tier architecture of AgriSense AI. Periods and rate limits are the deployed firmware constants. The irrigation decision executes on the sensor node and remains authoritative when the cloud tier is unreachable.

Four design principles shaped the implementation:

- **P1 – Edge authority.** The device makes the final pump decision; cloud components may only *request* actuation.
- **P2 – Fail-safe defaults.** Any missing, stale or implausible input resolves to *pump off*.
- **P3 – Least privilege.** Clients never hold third-party API keys or the database service key; every table is scoped to its owner by row-level security.
- **P4 – Graceful degradation.** Each optional service (diagnosis, advisor, camera) returns HTTP 503 when its credential is absent, without affecting the others.

The decisive architectural choice is that the sensor node talks **directly** to the database over HTTPS rather than through the backend. The backend is required only for scheduled automation, diagnosis, the advisor and the camera relay; telemetry, manual pump control and device pairing keep working with it stopped. Table 2 lists the data model.

TABLE 2. Database schema. All tables are protected by row-level security policies.

Table	Purpose
devices	Device ownership; trigger enforces ≤ 4 devices per account
device_registry	Factory serial and pairing secret
device_status	Heartbeat (last_seen_at), per-sensor health flags
device_config	Automation mode, thresholds, pump state
telemetry_data	Time series: temperature, humidity, soil, rain
device_commands	Command queue (PUMP_ON / PUMP_OFF), executed flag
timers	Irrigation windows: start, duration, active days

IV. HARDWARE IMPLEMENTATION

Fig. 2 and Table 3 give the sensing and actuation topology. Air temperature and relative humidity are read from a DHT11; soil moisture from a capacitive probe on a 12-bit ADC1 channel; rain from an active-low digital module; and a single relay with a normally-open contact switches the pump.

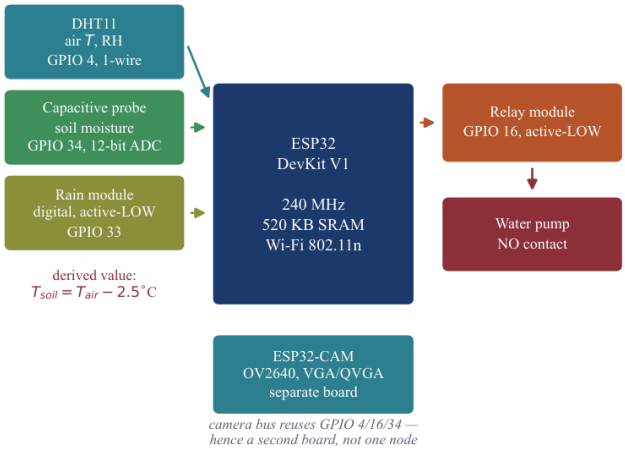


FIGURE 2. Sensor node wiring and pin assignment. Root-zone temperature is derived, not measured.

TABLE 3. Sensor node bill of materials and pin map (firmware v3.1.0).

Component	GPIO	Notes
DHT11 (air T, RH)	4	10 kΩ pull-up; 3 reads, 2.2 s apart
Capacitive soil probe	34	ADC1, input-only; dry 4095, wet 30
Rain module (DO)	33	Active-LOW; cannot be health-checked
Relay (pump)	16	Active-LOW, NO contact
ESP32-CAM (OV2640)	—	Separate board; VGA/QVGA JPEG

A. WHY TWO BOARDS

On the AI-Thinker ESP32-CAM the OV2640 interface and PSRAM occupy GPIO 34 (the soil ADC), GPIO 4 (flash LED) and GPIO 16 (PSRAM chip-select), leaving no ADC1 pin free for the soil probe. Imaging is therefore placed on a second board that shares only the Wi-Fi network. This also isolates the safety-critical control loop from camera memory pressure.

B. SENSOR CONDITIONING

The averaged raw ADC value r is mapped to a moisture percentage by a two-point calibration measured on the deployed probe, with dry-air reading $D = 4095$ and saturated-soil reading $W = 30$:

$$m(r) = \frac{D - r}{D - W} \times 100, \qquad r \geq r_{\text{disc}} = 10 \tag{1}$$

Readings below $r_{\text{disc}} = 10$ are rejected as a disconnected probe instead of being converted, because an unplugged input floats to approximately zero, which (1) would otherwise map to above 100 % and be indistinguishable from flooded soil. The DHT11 is retried up to three times per tick, 2.2 s apart (beyond the library's 2 s cache window); on failure the node reports the last good reading rather than zeros. Soil temperature is a derived quantity (air temperature – 2.5 °C) and is labelled as such. The rain module cannot be health-checked: a floating input reads as "no rain", which is fail-safe for watering only because rain merely *inhibits* the pump.

V. FIRMWARE AND ON-NODE CONTROL

A. TASK SCHEDULE

The firmware (Arduino C++, version 3.1.0) runs a cooperative loop with four periodic tasks: telemetry upload every 10 s, heartbeat upsert to *device_status* every 10 s, command polling every 5 s, and a pairing check every 30 s. The heartbeat carries per-sensor health flags so the backend can gate automation per sensor.

B. ORDERED IRRIGATION POLICY

Let m be calibrated moisture, $s \in \{0,1\}$ probe health, $\rho \in \{0,1\}$ rain, $u \in \{-1,0,+1\}$ the pending manual command and p the current pump state. Algorithm 1 is evaluated every 10 s; the first matching rule wins, so the three sensor interlocks always dominate operator intent: a manual *run* cannot water through rain, wet soil or a failed probe.

Algorithm 1: pumpShouldRun(m, s, ρ, p, u)

```
if s = 0      then return OFF  // failed probe
if ρ = 1      then return OFF  // raining
if m ≥ θ_wet then return OFF  // wet (70 %)
if u = -1     then return OFF  // force stop
if u = +1     then return ON   // manual run
if m < θ_dry then return ON   // dry (30 %)
return p                      // band: hold
```

The last rule implements the hysteresis band of Fig. 3, which prevents relay chatter when moisture hovers near a single threshold. Two hard limits bound delivery regardless of sensor state,

$$\tau_{\text{run}} \leq 15 \text{ min}, \qquad \tau_{\text{cool}} \geq 30 \text{ min}, \tag{2}$$

so a mis-calibrated probe cannot cycle the pump indefinitely. A manual command expires after 10 min and control returns to the automatic rules.

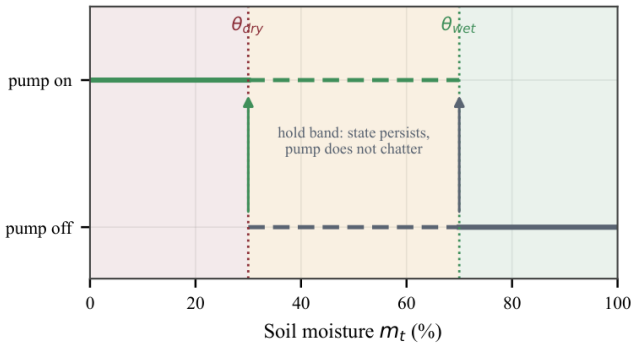


FIGURE 3. Hysteresis characteristic: between θ_dry = 30 % and θ_wet = 70 % the pump keeps its previous state.

C. BOOT-TIME SELF-TEST

Inverted comparisons or swapped thresholds would produce a pump that never stops, a fault that is costly to find in the field. The firmware therefore asserts Algorithm 1 against 11 hand-written cases at every boot with no hardware attached (Table 4). We additionally ported the function line-by-line to Python and verified the same 11 cases before using it in the simulation of Section X-D.

TABLE 4. Firmware boot self-test cases for Algorithm 1 (all 11 pass).

#	m (%)	s	ρ	p	u	Expected	Scenario
1	10	1	0	0	0	ON	dry + idle → start
2	10	1	1	0	0	OFF	dry but raining
3	10	0	0	1	0	OFF	probe unplugged
4	45	1	0	1	0	ON	in band, running → hold
5	45	1	0	0	0	OFF	in band, idle → hold
6	80	1	0	1	0	OFF	watered → stop
7	10	1	0	0	+1	ON	manual ON, dry
8	10	1	1	1	+1	OFF	rain beats manual ON
9	90	1	0	1	+1	OFF	wet beats manual ON
10	10	1	0	1	−1	OFF	manual OFF on dry soil
11	10	0	0	1	+1	OFF	unplugged beats manual ON

VI. CLOUD BACKEND

A. SUPERVISORY AUTOMATION ENGINE

The backend runs an asynchronous loop every 10 s over all devices whose automation mode is not NONE. The engine is stateless: mode, thresholds and pump state are re-read from *device_config* on each tick, so a restart loses nothing. Before evaluating a mode, a safety gate blocks automation and queues PUMP_OFF if the device is offline or (in moisture mode) the data are stale or the soil probe reports a fault, with

$$\text{offline} \Leftrightarrow t - t_{\text{seen}} > 30 \text{ s}, \qquad \text{stale} \Leftrightarrow t - t_{\text{telemetry}} > 60 \text{ s}. \tag{3}$$

The gate is per mode: timer mode reads no sensor, so a failed DHT11 must not disable it. Two modes are offered. **Moisture mode** starts the pump when soil is below a user start threshold and it is not raining, stops it at a stop threshold, and enforces a configurable maximum runtime (default 20 min). **Timer mode** runs the pump within user-defined windows (start time, duration, weekdays), pausing during rain; it is also the fixed-schedule control arm for a future metered trial. Commands are only *queued* in *device_commands*; the firmware treats them as manual requests subject to Algorithm 1, so the cloud can never override the on-node interlocks.

B. API SURFACE AND SECURITY

TABLE 5. Backend REST endpoints.

Endpoint	Function	Auth
POST /api/command	Manual pump ON/OFF, force stop	JWT
POST /api/automation/mode	NONE / MOISTURE / TIMER	JWT
/api/timers (CRUD)	Irrigation schedules	JWT
POST /api/plant/predict	Leaf diagnosis	JWT
POST /api/advisor/ask	Grounded advisory chat	JWT
POST /api/camera/{id}/frame	Upload JPEG frame	X-Cam-Key
GET /api/camera/{id}/wanted	Is a viewer attached?	X-Cam-Key
POST /api/camera/{id}/token	Short-lived viewer token	JWT
GET /api/camera/stream	MJPEG stream	Token

User tokens are validated on every request, and a malformed token yields HTTP 401 rather than a server error. Row-level security restricts every table to its owner, and a database trigger limits each account to four devices. A device is paired with its factory serial and secret and only while it is online (fresh heartbeat), which prevents claiming a device from a leaked label alone. API keys for the vision model, the LLM and the model server exist only in the backend environment.

C. DEMAND-DRIVEN CAMERA RELAY

The ESP32-CAM polls */wanted* every 3 s while idle. When a client requests a viewer token, the camera begins posting JPEG frames, capped at 4 frames/s, at VGA with quality 12 when PSRAM is present or QVGA with quality 15 otherwise; the backend re-emits them to viewers as an MJPEG stream. Because both camera and viewer make outbound requests to the backend, the feed works across NAT from any network without port forwarding, and the camera consumes upstream bandwidth only while watched.

VII. PLANT DISEASE DIAGNOSIS PIPELINE

The diagnosis subsystem (Fig. 4) is a Python package with seven unit-tested stages: acquisition, structural inspection, cleaning, taxonomy unification, manifest building, splitting and training, followed by ONNX export, verification and a serving runtime.



FIGURE 4. Transfer-learning and export pipeline. Values are those of the completed baseline run.

A. DATASETS

TABLE 6. Datasets configured in the pipeline (image counts are those of the configured mirrors). PlantDoc is never merged into training.

Dataset	Images	Classes	Imagery	Role
PlantVillage [20]	54,305	38	Lab	Train (baseline)
PlantWild [34]	30,030	—	Web/field	Train (planned)
Cassava [35]	21,367	5	Field	Train (planned)
Rice leaf [36]	5,932	4	Field	Train (planned)
PlantDoc [22]	2,598	27	Field	**Evaluation only**

B. CLEANING AND DE-DUPLICATION

Each image is decoded, size-checked and fingerprinted with SHA-256 and a 64-bit perceptual hash (pHash) [37]. Exact duplicates are removed by SHA-256. Near-duplicates are grouped when the Hamming distance between their hashes satisfies

$$d_H(h_i, h_j) = \sum_{k=1}^{64} \left[h_i^{(k)} \neq h_j^{(k)} \right] \leq \tau, \qquad \tau = 5, \tag{4}$$

with candidates bucketed by a 4-hex-digit hash prefix to keep comparisons tractable at ~10^5 images. Any training image within τ of a PlantDoc image is rejected, so the held-out field set cannot leak into training through re-encoded copies.

C. SPLITTING

Splits are stratified per class with target ratios 80/10/10 (tolerance ±2 %) and are **duplicate-group aware**: all members of a near-duplicate group are assigned to the same partition, which removes the leakage path described in [15].

D. TWO-STAGE MODEL

Stage 1 classifies species (YOLO11s-cls); Stage 2 uses one smaller YOLO11n-cls per species to classify the condition. The model is trained by minimising categorical cross-entropy

$$\mathcal{L} = -\frac{1}{N} \sum_{n=1}^{N} \sum_{i=1}^{C} y_{n,i} \log \hat{y}_{n,i} \tag{5}$$

with a cosine learning-rate schedule [38], mixed precision and early stopping (patience 10 for Stage 1, 8 for Stage 2). Augmentation uses HSV jitter (h 0.015, s 0.4, v 0.4), rotation up to 10°, horizontal flip with probability 0.5 and random erasing 0.2 [39]; vertical flips are disabled because leaves have a natural orientation. At inference the joint confidence and a low-confidence flag are

$$c = c_s \, c_c, \qquad \text{low} \Leftrightarrow c_s < 0.5 \ \vee \ c_c < 0.5, \tag{6}$$

where c_s and c_c are the species and condition soft-max confidences; thresholds are read from a versioned *registry.json*, never hard-coded in the server.

E. EXPORT AND SERVING

Models are exported to ONNX (opset 12), verified numerically against PyTorch, and served by a separate FastAPI service with ONNX Runtime [40] behind an API key. The backend selects the inference source at request time: the trained model service when it is configured, otherwise a hosted vision-language model (Gemini) constrained by a JSON response schema (fields: is_plant, species, condition, both confidences, healthy, severity ∈ {none, mild, moderate, severe}, affected area %, symptoms, cause, treatment, prevention) at temperature 0.2. Swapping in the trained model is an environment change, not a code change.

VIII. TELEMETRY-GROUNDED FARM ADVISOR

The advisor endpoint fetches the device's most recent telemetry and places it in the LLM system instruction, marking it *STALE* when older than the freshness window, so answers refer to measured field conditions rather than generic advice. The plant-scan screen can hand a diagnosis directly into the chat. In the current implementation both the advisor and the vision fallback call one hosted provider directly; a provider-neutral adapter and an offline fallback are future work.

IX. CLIENT APPLICATIONS

The web dashboard is plain HTML/CSS/JavaScript served by the backend and subscribes to database changes through Realtime. The mobile application is written in TypeScript with React Native, Expo 54 and expo-router, with four tabs: Dashboard (live tiles, pump control, automation mode, timers, force stop), Plant Scan, Advisor and Account (Fig. 5).

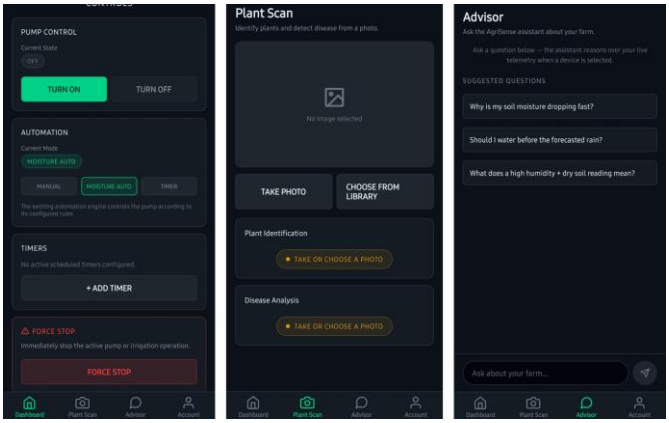


FIGURE 5. Mobile application: irrigation controls (left), plant scan (centre) and farm advisor (right).

X. EXPERIMENTAL RESULTS

A. TRAINING SETUP

The baseline is a single-stage YOLO11n-cls classifier over the 38 PlantVillage species–condition classes (14 crops), initialised from pretrained weights and fine-tuned for 5 epochs at 224×224, batch 64, seed 42, deterministic mode, on a Google Colab T4 GPU. The split is 43,444 training and 10,861 validation images. Total training time was 1,300 s (21.7 min).

TABLE 7. Per-epoch training log (results.csv). Time is cumulative.

Epoch	Train loss	Val loss	Top-1 (%)	Top-5 (%)	Time (s)
1	1.689	0.210	94.75	99.82	265.9
2	0.198	0.071	97.87	99.97	524.7
3	0.109	0.038	98.91	100.00	780.5
4	0.075	0.028	99.21	100.00	1040.4
5	0.052	0.024	**99.32**	**100.00**	1300.0

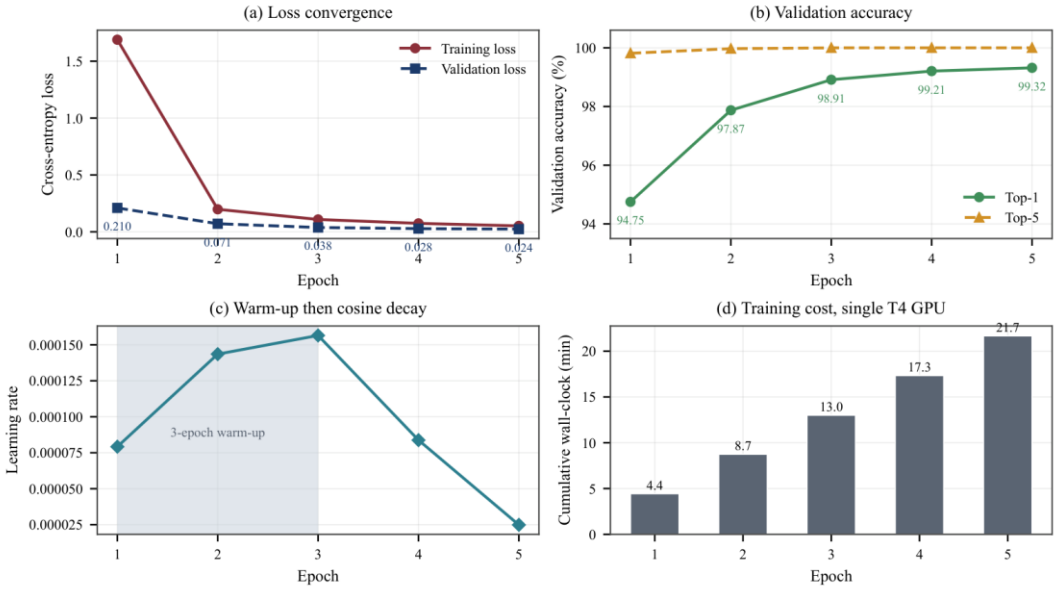


FIGURE 6. Training and validation loss and accuracy over five epochs.

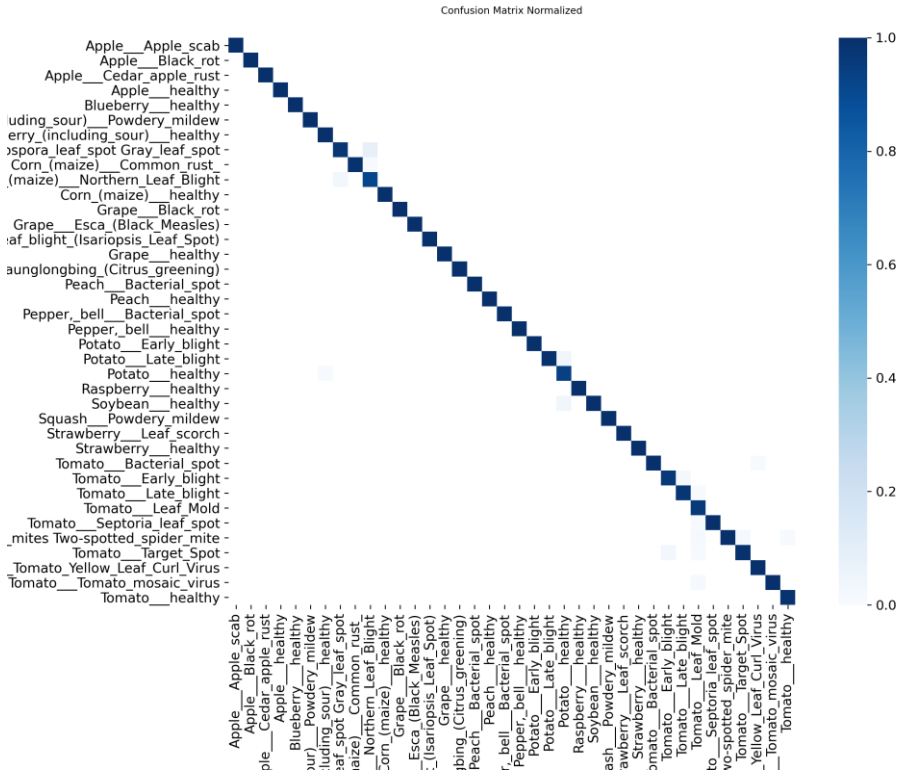


FIGURE 7. Row-normalised confusion matrix over the 38 classes on the validation split.

B. MODEL COMPLEXITY AND SERVER-SIDE LATENCY

The exported model has 1,574,708 parameters (counted from the ONNX initializers) in 155 graph nodes; the PyTorch checkpoint is 3.13 MiB and the FP32 ONNX file 6.10 MiB. Latency was measured with ONNX Runtime 1.31 on an AMD Ryzen 5 5625U laptop CPU, batch 1, 224×224 input, 20 warm-up runs followed by 200 timed runs (Table 8). A single CPU thread suffices for interactive diagnosis.

TABLE 8. ONNX Runtime CPU inference latency (n = 200).

Threads	Mean (ms)	Std (ms)	p50 (ms)	p95 (ms)
1	6.07	1.18	5.81	8.36
4	3.98	1.77	3.49	6.78

C. STATIC EDGE-FEASIBILITY ANALYSIS

For an input of side r, c channels and b bytes per element the input activation alone needs

$$M_{\text{act}} = r^2\, c\, b = 224^2 \times 3 \times 4\ \text{B} = 588\ \text{KiB}, \tag{7}$$

which already exceeds the 520 KiB SRAM of the base ESP32 [41] before any intermediate feature map, camera frame buffer or Wi-Fi stack is allocated. The FP32 model is 12.0× that SRAM budget, and even an ideal 4× INT8 compression [27] leaves 1.53 MiB (Fig. 8). On-device inference therefore requires a smaller input, fewer classes, PSRAM-backed tensor arenas or an ESP32-S3-class part; the deployed design runs inference on the server.

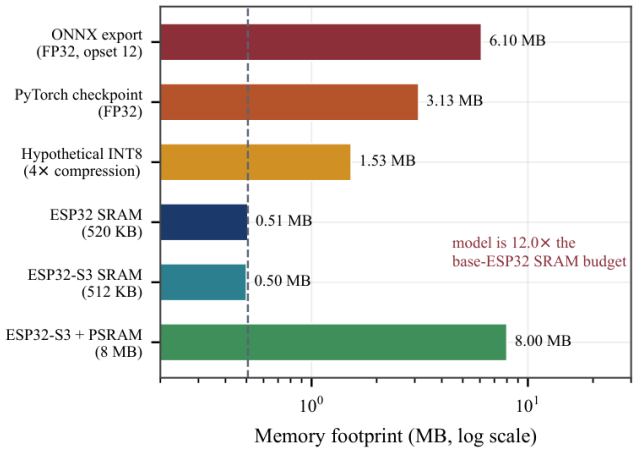


FIGURE 8. Model footprint versus on-chip memory budgets (log scale).

D. CONTROLLER SIMULATION

To illustrate why the hold rule matters, we simulated 48 h with a bucket soil model: moisture falls 2 %/h by evapotranspiration, rises 0.12 % per 10 s tick while pumping, and one hour of rain occurs at t = 20 h; the sensor adds Gaussian noise (σ = 2 %), and the 15 min cap and 30 min cooldown apply to both rules. These parameters are assumptions, not field measurements. A single 30 % threshold switched the relay **1292** times as noise crossed the threshold, whereas Algorithm 1 switched **16** times for similar pump-on time (114 vs. 120 min) (Fig. 9). The simulation also exposed a design interaction: with this wetting rate the 15 min cap ends each cycle near 45 %, well short of θ_wet, so in

practice the cap rather than the upper threshold bounds each irrigation event.

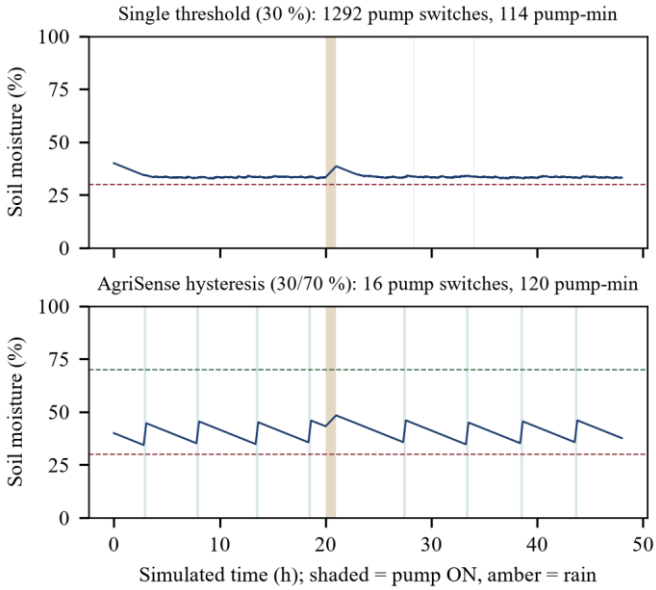


FIGURE 9. Simulated 48 h trace (assumed soil model; not a field result). Hysteresis reduces relay switching by roughly two orders of magnitude under 2 % sensor noise.

E. FUNCTIONAL VERIFICATION

TABLE 9. Automated verification executed for this paper.

Suite	Scope	Result
Firmware self-test	Algorithm 1, 11 cases	11/11 pass
ML unit tests (pytest)	Cleaning, manifest, split, taxonomy, config, pipeline	31/31 pass
Backend test_automation	Per-mode safety gate	Pass
Backend test_plant_predict	Diagnosis response normalisation	Pass
Python port of Algorithm 1	Same 11 cases	11/11 pass

XI. DISCUSSION AND LIMITATIONS

The 99.32 % is a laboratory upper bound. The baseline was trained with a naïve per-class 80/20 random split, not the duplicate-group-aware split of Section VII, so validation accuracy may be inflated by near-duplicates. PlantVillage photographs are single detached leaves on plain backgrounds; published models trained on it fall to 31–68 % on field images (Fig. 10). The field gap of this model has not yet been measured.

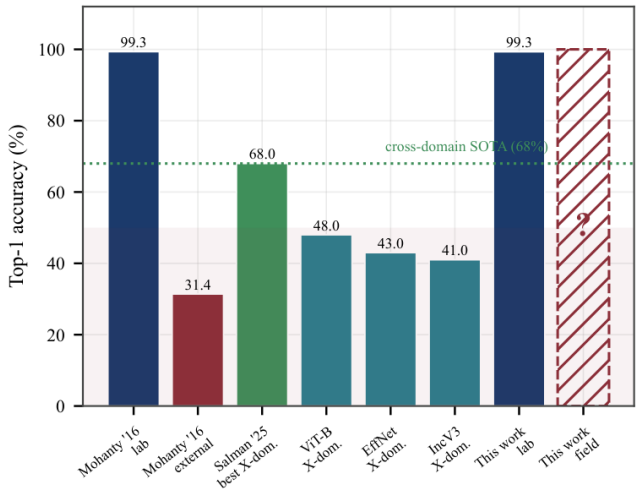


FIGURE 10. Published laboratory-to-field accuracy evidence [8], [14]; the field value for this work is pending.

Further limitations, stated so they are not mistaken for capabilities:

- The deployed Plant Scan currently uses the vision-language fallback; the full two-stage pipeline is implemented but has not been trained end to end (it needs ~13 GB of data and more GPU time than a free Colab session).
- No inline flow meter is installed, so no water-saving percentage is claimed.
- The firmware does not report measured pump state, so cloud and node can briefly disagree.
- The DHT11 has limited accuracy (±2 °C, ±5 % RH class), and soil temperature is derived.
- The LLM layer is bound to one hosted provider and fails closed (HTTP 503) when unreachable.
- Per-unit cost, on-device latency, power, uptime and packet loss have not been measured.

The planned next steps are: evaluation on PlantDoc and on a hand-labelled set captured by the project's own ESP32-CAM; a retrain on the de-duplicated split to quantify leakage; a two-plot, flow-metered trial of moisture mode against timer mode; and an INT8 feasibility sweep on ESP32-S3.

XII. CONCLUSION

AgriSense AI shows that the familiar sensor-plus-camera-plus-classifier architecture can be built around its failure modes rather than its happy path. Placing an ordered, self-tested rule set on the node keeps irrigation safe and available when the cloud is not; routing telemetry directly to a secured database makes every value-added service optional; and a data pipeline that removes near-duplicates and quarantines field data makes the reported accuracy auditable. The measured baseline (99.32 % top-1 on laboratory images, 6.07 ms per image on one CPU thread) proves the training-to-serving path end to end. We have deliberately not claimed field accuracy or water savings; the protocol and code to measure both are in place, and the full source is released.

ACKNOWLEDGMENT

The authors thank [Guide Name] and the Department of Emerging Technologies, S. B. Jain Institute of Technology, Management and Research, Nagpur, for guidance and the maintainers of PlantVillage, PlantDoc, Ultralytics, ONNX Runtime and Supabase for openly available datasets and tools.

REFERENCES

[1] Food, and Agriculture Organization of the United Nations, "AQUASTAT: FAO's Global Information System on Water and Agriculture," 2024. [Online]. Available: https://www.fao.org/aquastat/.

[2] S. Savary, L. Willocquet, S. J. Pethybridge, P. Esker, N. McRoberts, and A. Nelson, "The Global Burden of Pathogens and Pests on Major Food Crops," *Nature Ecology & Evolution*, vol. 3, no. 3, pp. 430–439, 2019, doi: 10.1038/s41559-018-0793-y.

[3] R. N. Strange, and P. R. Scott, "Plant Disease: A Threat to Global Food Security," *Annual Review of Phytopathology*, vol. 43, pp. 83–116, 2005, doi: 10.1146/annurev.phyto.43.113004.133839.

[4] J. Gutiérrez, J. F. Villa-Medina, A. Nieto-Garibay, and M. Porta-G\'andara, "Automated Irrigation System Using a Wireless Sensor Network and GPRS Module," *IEEE Transactions on Instrumentation and Measurement*, vol. 63, no. 1, pp. 166–176, 2014, doi: 10.1109/TIM.2013.2276487.

[5] A. Goap, D. Sharma, A. K. Shukla, and C. Rama Krishna, "An IoT Based Smart Irrigation Management System Using Machine Learning and Open Source Technologies," *Computers and Electronics in Agriculture*, vol. 155, pp. 41–49, 2018, doi: 10.1016/j.compag.2018.09.040.

[6] C. Kamienski et al., "Smart Water Management Platform: IoT-Based Precision Irrigation for Agriculture," *Sensors*, vol. 19, no. 2, Art. no. 276, 2019, doi: 10.3390/s19020276.

[7] L. García, L. Parra, J. M. Jimenez, J. Lloret, and P. Lorenz, "IoT-Based Smart Irrigation Systems: An Overview on the Recent Trends on Sensors and IoT Systems for Irrigation in Precision Agriculture," *Sensors*, vol. 20, no. 4, Art. no. 1042, 2020, doi: 10.3390/s20041042.

[8] S. P. Mohanty, D. P. Hughes, and M. Salathé, "Using Deep Learning for Image-Based Plant Disease Detection," *Frontiers in Plant Science*, vol. 7, Art. no. 1419, 2016, doi: 10.3389/fpls.2016.01419.

[9] K. P. Ferentinos, "Deep Learning Models for Plant Disease Detection and Diagnosis," *Computers and Electronics in Agriculture*, vol. 145, pp. 311–318, 2018, doi: 10.1016/j.compag.2018.01.009.

[10] E. C. Too, L. Yujian, S. Njuki, and L. Yingchun, "A Comparative Study of Fine-Tuning Deep Learning Models for Plant Disease Identification," *Computers and Electronics in Agriculture*, vol. 161, pp. 272–279, 2019, doi: 10.1016/j.compag.2018.03.032.

[11] R. Samanta, B. Saha, and S. K. Ghosh, "LeafSense: A Portable, Low-Cost, Low-Power Plant Disease Diagnostic Device Using TinyML," in *Proc. 4th Int. Conf. AI-ML Systems*, 2024, doi: 10.1145/3703412.3703438.

[12] J. P. Nyakuri, C. Nkundineza, O. Gatera, K. Nkurikiyeyezu, and G. Mwitende, "AI and IoT-Powered Edge Device Optimized for Crop Pest and Disease Detection," *Scientific Reports*, vol. 15, Art. no. 22905, 2025, doi: 10.1038/s41598-025-06452-5.

[13] H. Aqasizade, M. Antonini, M. Vecchio, and F. Antonelli, "A TinyMLOps Pipeline for Coarse-Grained Plant Disease Classification in Precision Agriculture," *Sensors*, vol. 26, no. 16, Art. no. 5029, 2026, doi: 10.3390/s26165029.

[14] Z. Salman, A. Muhammad, and D. Han, "Plant Disease Classification in the Wild Using Vision Transformers and Mixture of Experts," *Frontiers in Plant Science*, vol. 16, Art. no. 1522985, 2025, doi: 10.3389/fpls.2025.1522985.

[15] S. Kapoor, and A. Narayanan, "Leakage and the Reproducibility Crisis in Machine-Learning-Based Science," *Patterns*, vol. 4, no. 9, Art. no. 100804, 2023, doi: 10.1016/j.patter.2023.100804.

[16] D. Vallejo-Gómez, M. Osorio, and C. A. Hincapié, "Smart Irrigation Systems in Agriculture: A Systematic Review," *Agronomy*, vol. 13, no. 2, Art. no. 342, 2023, doi: 10.3390/agronomy13020342.

[17] K. Usman, and A. Zubairu, "The Use of AI in IoT-Based Smart Irrigation," *CEDTECH International Journal of Natural and Applied Sciences*, vol. 7, no. 1, pp. 57–72, 2026, doi: 10.5281/zenodo.20611846.

[18] J. Deng, W. Dong, R. Socher, L. Li, K. Li, and L. Fei-Fei, "ImageNet: A Large-Scale Hierarchical Image Database," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2009, pp. 248–255, doi: 10.1109/CVPR.2009.5206848.

[19] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778, doi: 10.1109/CVPR.2016.90.

[20] D. P. Hughes, and M. Salathé, "An Open Access Repository of Images on Plant Health to Enable the Development of Mobile Disease Diagnostics," *arXiv preprint arXiv:1511.08060*, 2015.

[21] M. Brahimi, K. Boukhalfa, and A. Moussaoui, "Deep Learning for Tomato Diseases: Classification and Symptoms Visualization," *Applied Artificial Intelligence*, vol. 31, no. 4, pp. 299–315, 2017, doi: 10.1080/08839514.2017.1315516.

[22] D. Singh, N. Jain, P. Jain, P. Kayal, S. Kumawat, and N. Batra, "PlantDoc: A Dataset for Visual Plant Disease Detection," in *Proc. 7th ACM IKDD CoDS and 25th COMAD*, 2020, pp. 249–253, doi: 10.1145/3371158.3371196.

[23] J. G. A. Barbedo, "Factors Influencing the Use of Deep Learning for Plant Disease Recognition," *Biosystems Engineering*, vol. 172, pp. 84–91, 2018, doi: 10.1016/j.biosystemseng.2018.05.013.

[24] G. Jocher, and J. Qiu, "Ultralytics YOLO11," 2024. [Online]. Available: https://github.com/ultralytics/ultralytics.

[25] J. Lin, W. Chen, Y. Lin, J. Cohn, C. Gan, and S. Han, "MCUNet: Tiny Deep Learning on IoT Devices," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[26] J. Lin, W. Chen, H. Cai, C. Gan, and S. Han, "MCUNetV2: Memory-Efficient Patch-Based Inference for Tiny Deep Learning," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.

[27] B. Jacob et al., "Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 2704–2713, doi: 10.1109/CVPR.2018.00286.

[28] R. David et al., "TensorFlow Lite Micro: Embedded Machine Learning for TinyML Systems," in *Proc. Machine Learning and Systems (MLSys)*, 2021, pp. 800–811.

[29] A. Tzachor et al., "Large Language Models and Agricultural Extension Services," *Nature Food*, vol. 4, pp. 941–948, 2023, doi: 10.1038/s43016-023-00867-x.

[30] B. Silva, L. Nunes, R. Estação, V. Aski, and R. Chandra, "GPT-4 as an Agronomist Assistant? Answering Agriculture Exams Using Large Language Models," *arXiv preprint arXiv:2310.06225*, 2023.

[31] A. Balaguer et al., "RAG vs Fine-Tuning: Pipelines, Tradeoffs, and a Case Study on Agriculture," *arXiv preprint arXiv:2401.08406*, 2024.

[32] P. Lewis, E. Perez, A. Piktus et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020, pp. 9459–9474.

[33] Supabase, "Supabase: The Postgres Development Platform," 2026. [Online]. Available: https://supabase.com/docs.

[34] T. Wei et al., "PlantWild: An In-the-Wild Plant Disease Recognition Dataset," 2024. [Online]. Available: https://huggingface.co/datasets/uqtwei2/PlantWild.

[35] E. Mwebaze, T. Gebru, A. Frome, S. Nsumba, and J. Tusubira, "iCassava 2019 Fine-Grained Visual Categorization Challenge," *arXiv preprint arXiv:1908.02900*, 2019.

[36] N. Sankalana, "Rice Leaf Disease Image Samples," 2023. [Online]. Available: Kaggle dataset, https://www.kaggle.com/datasets/nirmalsankalana/rice-leaf-disease-image.

[37] C. Zauner, "Implementation and Benchmarking of Perceptual Image Hash Functions," *Master's thesis, Upper Austria University of Applied Sciences, Hagenberg*, 2010.

[38] I. Loshchilov, and F. Hutter, "SGDR: Stochastic Gradient Descent with Warm Restarts," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2017.

[39] Z. Zhong, L. Zheng, G. Kang, S. Li, and Y. Yang, "Random Erasing Data Augmentation," in *Proc. AAAI Conf. Artificial Intelligence*, 2020, pp. 13001–13008, doi: 10.1609/aaai.v34i07.7000.

[40] ONNX Runtime developers, "ONNX Runtime: Cross-Platform, High Performance ML Inferencing and Training Accelerator," 2026. [Online]. Available: https://onnxruntime.ai/.

[41] Espressif Systems, "ESP32 Series Datasheet," 2026. [Online]. Available: https://www.espressif.com/en/support/documents/technical-documents.

TISHA JHARIYA is with the Department of Emerging Technologies (Artificial Intelligence & Data Science), S. B. Jain Institute of Technology, Management and Research, Nagpur, India.

AKIKO GAJBHIYE is with the Department of Emerging Technologies (Artificial Intelligence & Data Science), S. B. Jain Institute of Technology, Management and Research, Nagpur, India.

ANSH MESHRAM is with the Department of Emerging Technologies (Artificial Intelligence & Data Science), S. B. Jain Institute of Technology, Management and Research, Nagpur, India.

AGYA CHOURASIA is with the Department of Emerging Technologies (Artificial Intelligence & Data Science), S. B. Jain Institute of Technology, Management and Research, Nagpur, India.

[GUIDE NAME] is with the Department of Emerging Technologies, S. B. Jain Institute of Technology, Management and Research, Nagpur, India. [Research interests.]